Vulnerabilidades críticas

Exposición, demostración y mitigación

- Broken Access Control
- Cross-Site Scripting + CSP
- Server-Side Request Forgery
- 15 minutos · OWASP Top 10:2021

EQUIPO · INGENIERÍA EN SOFTWARE

[Nombre completo 1] — [Matrícula 1]

[Nombre completo 2] — [Matrícula 2]

[Nombre completo 3] — [Matrícula 3]

Completar antes de entregar. Máximo 3 integrantes.

En cada tema: teoría → OWASP → código y exploit → corrección.

Una aplicación, dos versiones

- Vulnerable: 127.0.0.1:3000
- Mitigado: 127.0.0.1:3001
- Sesión: ana / demo123
- JavaScript / Node.js · Sin Internet al exponer

FLUJO DE LA DEMOSTRACIÓN

Navegador → aplicación web

Aplicación → catálogo local (:9100)

Aplicación → servicio interno simulado (:9101)

Notas, cuentas y secretos ficticios. Todo escucha en 127.0.0.1.

El servicio interno es una simulación local; no representa una red aislada real.

Broken Access Control

- Fallo al aplicar los permisos de un usuario.
- Autenticación: ¿quién eres?
- Autorización: ¿qué puedes hacer?
- Impacto: leer, modificar o borrar datos ajenos.

EJEMPLO: IDOR

Ana inicia sesión correctamente.

GET /notas/1 → nota de Ana

Cambia el identificador:

GET /notas/2 → nota de Bruno

El ID selecciona el objeto; no concede permiso.

OWASP A01:2021 · Validar permisos por objeto y por operación en el servidor.

Cambiar un ID expone otra nota

- Abrir :3000/notas/1 como Ana.
- Cambiar el último número a 2.
- Resultado: HTTP 200 y presupuesto de Bruno.
- Falta comprobar la propiedad del objeto.

vulnerable/app.js · obtenerNota (fragmento)

```
const nota = notas[id];
if (!nota) {
  return { estado: 404,
           error: 'Nota inexistente' };
}

// No comprueba al propietario.
return { estado: 200, nota };
```

Demostración: una sesión válida accede a información ajena.

Autorizar antes de devolver el objeto

- La identidad viene de la sesión del servidor.
- Comprobar el propietario de cada nota.
- Ana + nota 1 → HTTP 200.
- Ana + nota 2 → HTTP 403.

mitigado/app.js · obtenerNota (fragmento)

```
if (nota.propietario !== usuario) {
  return {
    estado: 403,
    error: 'Acceso denegado: esta nota '
      + 'pertenece a otro usuario.'
  };
}

return { estado: 200, nota };
```

Ocultar botones o usar UUID no sustituye la autorización en backend.

XSS: datos que el navegador ejecuta

- Entrada no confiable se interpreta como HTML ejecutable.
- Tipos: reflejado, almacenado y basado en DOM.
- Nuestra demo: XSS reflejado.
- Impacto: alterar la página o actuar con la sesión de la víctima.

DEL DATO AL CÓDIGO

Usuario envía un comentario.

El servidor lo inserta sin codificar.

El navegador crea una etiqueta HTML.

Un evento ejecuta JavaScript bajo el origen de la aplicación.

OWASP A03:2021 · Injection incluye XSS (CWE-79).

Un comentario provoca una alerta

- Abrir :3000/xss.
- Enviar el comentario precargado.
- La imagen falla al cargar y ejecuta onerror.
- El alert es una prueba inocua de ejecución.

vulnerable/app.js + entrada de prueba

```
function mostrarComentario(texto) {
  return texto; // Se inserta en un div.
}

// Entrada en el formulario:
<img src=x
  onerror="alert('XSS de laboratorio')">
```

No hace falta una etiqueta <script> para ejecutar JavaScript.

Codificar la salida y añadir CSP

- En :3001/xss, la entrada se muestra como texto.
- Escape contextual: < se representa como <.
- CSP es una defensa adicional.
- En /csp-prueba: comparar alertas y abrir F12 → Console.

mitigado/app.js · salida y directivas clave

```
function mostrarComentario(texto) {
  return escaparHTML(texto);
}

Content-Security-Policy:
  default-src 'none';
  script-src 'none';
  style-src 'self';
  object-src 'none';
  base-uri 'none'
```

La app no necesita scripts. CSP no sustituye el escape ni la sanitización contextual.

SSRF: el servidor hace la petición

- El usuario controla un destino consultado por el backend.
- Se aprovecha la conectividad del servidor.
- Impacto: acceso a servicios internos o metadatos.
- La demo utiliza un secreto totalmente ficticio.

¿QUIÉN SE CONECTA?

Usuario → URL elegida

Aplicación → petición HTTP al destino

Servicio interno → respuesta a la aplicación

Aplicación → datos devueltos al usuario

OWASP A10:2021 · En SSRF actúa el servidor; en XSS actúa el navegador.

De consultar un catálogo a leer un secreto

- Abrir :3000/ssrf.
- Consultar :9100/catalogo → productos.
- Cambiar por :9101/secreto.
- Resultado: SECRETO-DE-LABORATORIO-123.

vulnerable/app.js · consultarRecurso (fragmento)

```
// La URL proviene del usuario.
return descargar(url, {
  redirect: 'follow'
});

// Catálogo legítimo:
http://127.0.0.1:9100/catalogo

// Servicio interno simulado:
http://127.0.0.1:9101/secreto
```

Permitir HTTP/HTTPS y añadir un timeout no autoriza el destino.

Destinos fijos, sin redirecciones

- Allowlist exacta: esquema, IP, puerto y ruta.
- IP literal: sin resolución DNS en esta demo.
- Catálogo permitido → HTTP 200.
- Servicio interno → HTTP 403.
- Complemento real: restricciones de salida de red.

mitigado/app.js · consultarRecurso (fragmento)

```
if (!destinosPermitidos.has(url)) {
  const error = new Error(
    'Destino bloqueado'
  );
  error.status = 403;
  throw error;
}
const destinoFijo =
  destinosPermitidos.get(url);
return descargar(destinoFijo, {
  redirect: 'error'
});
```

Con dominios: controlar IP efectiva, IPv4/IPv6, DNS rebinding y rangos reservados.

Tres límites de confianza

- Acceso roto: usuario → objeto ajeno.
- XSS: dato → código del navegador.
- SSRF: entrada → conexión del servidor.

CONTROL QUE CORRIGE CADA FALLA

Autorización por objeto en backend.

Salida codificada según el contexto + CSP.

Destinos permitidos y redirecciones bloqueadas.

Verificar siempre: ataque rechazado y uso legítimo conservado.

No confiar en la entrada del cliente para decidir permisos, código o destinos.

Prompt, resultados y revisión

- Herramienta: GPT-6 Astra mediante OpenCode.
- Contexto: consigna completa de la actividad.
- Resultados: código, presentación, guion y pruebas.
- Equipo: [anotar revisiones y adaptaciones reales].

INSTRUCCIÓN LITERAL UTILIZADA

“no importa la fecha. nomas haz lo que se necesita y dime que hacer o explicar.”

Adaptaciones del asistente: Node.js disponible, datos ficticios y servicios locales.

Registro ampliado: USO-IA.md. El equipo debe ensayar y explicar el código.

Documentar lo que realmente se utilizó y adaptó; citar fuentes técnicas oficiales.

Fuentes oficiales y entrega

- OWASP Top 10:2021: edición aceptada por la consigna.
- Código organizado en /vulnerable y /mitigado.
- Entrega: presentación + enlace al proyecto completo.
- Gracias. ¿Preguntas?

REFERENCIAS · ENLACES CLICABLES

OWASP A01:2021 · Broken Access Control

OWASP A03:2021 · Injection (incluye XSS)

OWASP A10:2021 · SSRF

OWASP · Authorization Cheat Sheet

OWASP · XSS Prevention Cheat Sheet

OWASP · Content Security Policy Cheat Sheet

OWASP · SSRF Prevention Cheat Sheet

Referencias completas: FUENTES.md · Plantilla de entrega: CORREO.md